Lesson Production Standards

The standing rules for building every lesson, 2 through 30

VIDEO GAME CREATION

Purpose

This document is the reference standard for producing materials for every remaining lesson in the course. It exists so that Lesson 2 and Lesson 29 feel like they came from the same course, and so lesson-building doesn't require re-deciding format and structure each week. Three content rules live here (the Three Levels of Accomplishment, the five required outputs for every lesson, and the self-containment requirement), plus the production standards that grew out of actually building Lessons 1 and 2: the shared visual branding system, the slide deck conventions, and the diagram conventions.

Rule 1: Three Levels of Accomplishment

This course does not use a numeric grading rubric. Because the class spans a wide range of ages and prior experience, every lesson's hands-on work - and both semester milestones - are structured around three levels instead. The goal: no student is ever stuck unable to move forward, and no advanced student is handed busy work to stay engaged.

Level	Think of it as...	What it means	Writing it into a handout
1 - Minimal	MVP	The smallest working version of the concept - enough to understand it and move on next week. A complete, legitimate finish, not a fallback.	Should be achievable in the lab time remaining even by a student who lost 20+ minutes to install trouble. One or two concrete, checkable steps.
2 - Complete	Alpha/Beta	All of the lesson's expected features are present and working end-to-end. Art, final polish, and edge-case bugs may still be missing.	Should match the lesson's full stated objective as written in the lesson plan. This is "done," not "done well."
3 - Refined	Publish-Ready	Feature-complete, tested, and polished enough to comfortably share outside class.	Write as a short menu of stretch goals/polish ideas, not one fixed bar - students choose what interests them.

Ground rules for applying this

- Level 1 is always a complete, legitimate finish for the lesson. Never phrase it as a consolation prize.
- Levels 2 and 3 are invitations, not requirements - never required for a student to move on to next week's lesson.
- Every Student Handout must state what Level 1, 2, and 3 look like for that lesson's specific exercise (see Rule 2 below).
- The Semester 1 Mini-Game and Semester 2 Expo game milestones use the same three levels, at a project scale instead of a single-lesson scale.
- Levels describe the same underlying exercise at different depths - they should not be three unrelated tasks.
- Unless there's a solid engine-specific reason not to, every engine's version of a given level should be doing the same underlying thing - same input, same action - not a different exercise that merely looks equivalent. If one engine's version ends up teaching something else, that's a sign to redesign it to match the others, not to document it as an acceptable quirk. (Lesson 2 caught exactly this: 7 engines used "press the right arrow key, move right" for Level 1, and one used a color change instead - the color-change engine was rewritten to match, and the color idea was kept as a Level 3 stretch goal instead, where variation is expected.)
- Before writing any engine-specific solution, state each level's goal as a plain-language, observable behavior - not an implementation. "Animate the sprite" is not concrete enough on its own; it leaves "animate" open to mean whatever the first engine happens to make easy. See "Defining a Lesson's Level 1/2/3 Goals" below for the framework and a worked example.

Defining a Lesson's Level 1/2/3 Goals

Write a lesson's Level 1/2/3 as a short list of observable, plain-language behaviors before touching any engine. Every engine's Handout and Key are then built to satisfy that same list - the description of what happens on screen (and in the speakers) should read the same regardless of which engine's document you're holding, even though the underlying code or visual-scripting graph achieving it will differ from engine to engine.

Worked example: Lesson 2

Lesson 2's exercise ("Key Press Reactor") uses this list. It's kept here as the reference example for how to write this section for future lessons:

- **Level 1:** Move the sprite right via a directional button.
- **Level 2:** Move the sprite right, left, up, and down via different, appropriate directional buttons.
- **Level 3:** Animate the sprite when in motion by making it blink while moving and stay static when not; add a beep-beep sound while moving, silent when not; change color while moving, using a different color per direction, returning to the original color when not; clamp movement to the screen; and structure the code so that everything triggered by the same action happens in the same place in the code.

Note how each Level 3 item is stated as a result, not a technique - "blink while moving" rather than "alternate two sprite costumes" or "toggle scale between 0 and 1." The technique is chosen per engine (see "Same behavior, not the same code" below).

Same behavior, not the same code

It is not necessary for the actual implementation to be identical across engines - a goal like "blink" or "change color" should be implemented in whichever way is simplest and most idiomatic for that specific engine. Scratch's version might swap sprite costumes; TIC-80's might toggle a scale argument to zero; another engine's might toggle a sprite's Visible property on a timer. That variation is expected and fine.

What does need to match, once every engine's solution is run side by side, is the observable behavior: they should all move the same way, blink the same way, beep the same way, and recolor the same way, on the same inputs. Code (or a visual-scripting graph) should follow that specific engine's own best practices while producing that same observable result - a Handout or Key that achieves the right look by fighting the engine's normal conventions needs a different approach, not a pass.

Assets Required

Apply this same default-first ordering when a Level 1/2/3 goal implies art or sound, so students spend lab time on the lesson's actual concept instead of hunting for or importing assets:

- Use that engine's own default/built-in sprite or shape if one exists and reasonably fits the goal.
- If no usable default exists - or the default doesn't lend itself well to the goal - use basic shapes/primitives instead of asking a student to import anything.
- Swapping in a supplied custom sprite is a fair Level 3 stretch idea; it is never a Level 1 or Level 2 requirement.
- Use that engine's own default sound editor if one exists and is reasonable for a student to pick up at this point in the course.
- If no sound editor exists, or using it would demand skills well beyond what's appropriate this early, fall back to a supplied sound effect asset instead (see Rule 3's self-containment requirement - any such asset must be pre-supplied, never something a student is sent to find).

Rule 2: Five Required Outputs Per Lesson

Every lesson (2 through 30) produces exactly five kinds of documents:

Output	Audience	Length
1-Page Lesson Plan & Outline	Instructor, quick reference	1 page, no exceptions - compact/dense formatting
Teacher's Script	Instructor, used live during lecture	As long as needed to cover the lecture segment; written to be spoken, not read as prose
Student Handout (one per engine)	Students, that engine's users	1-2 pages per engine version
Teacher's Checklist / Prep Guide	Instructor, used before class	As long as needed; task-list format

Output	Audience	Length
Teacher's Key (one per engine)	Instructor only - not distributed to students	Complete worked Level 1 and Level 2 solutions, plus every Level 3 idea fully worked out, per engine

A. 1-Page Lesson Plan & Outline

A compact instructor-facing summary - the thing to glance at five minutes before class. Fits on one page. Contains:

- Lesson number, topic, and date
- Objectives (2-4 bullet points)
- IGDA framework tie-in (one line)
- Featured engine for this week's live demo
- Materials needed (one line, cross-referencing the Teacher's Checklist for detail)
- Agenda with timing (lecture segments + lab), matching the format used in the Lesson 1 Plan
- One-line summary of that lesson's Level 1 / 2 / 3 (full detail lives in the Student Handouts)

B. Teacher's Script

Written to be spoken aloud during the lecture portion, organized to match the Lesson Plan's agenda segments. Not formal prose - closer to speaker notes. Contains:

- Talking points for each agenda segment, in delivery order
- Explicit demo cues in brackets, e.g. [Switch to demo laptop, open Godot, show the Input Map]
- Suggested discussion questions and prompts for student participation
- Timing checkpoints so the instructor can tell if a segment is running long
- A natural transition line into lab time

C. Student Handout (one per supported engine)

One version of the handout per engine the lesson is relevant to (up to all 8: Scratch, Godot, Unity, GameMaker, GDevelop, TIC-80, 001 Game Creator, Unreal). Each version follows the same template so students can swap engines without relearning the handout's structure. Contains:

- This lesson's topic and objective, restated for the student
- Step-by-step instructions for that lesson's exercise, in that specific engine's terms (menu paths, node/action names, code snippets as applicable)
- That lesson's Level 1 / 2 / 3 criteria, written specifically for this engine's version of the exercise
- Common troubleshooting tips specific to that engine
- A pointer to the matching entry in the Student Resource List for further practice

Note on scale: with up to 8 engine versions across 29 remaining lessons, this is a large volume of material. To keep it sustainable, handouts should be built from a shared template per lesson (same structure, section order, and Level 1/2/3 framing) so that only the engine-specific steps change between versions.

D. Teacher's Checklist / Prep Guide

A detailed, checkable setup guide the instructor runs through before class, so no engine surprises the instructor mid-lesson. Contains:

- What to pre-build and test in the featured engine before class (a working copy of that week's exercise, confirmed to run)
- Spot-checks for the other 7 engines if this lesson's concept touches something engine-specific (e.g., a rendering setting, an export step)
- Equipment/materials list (demo laptop, projector, handout copies, USB backups, etc.)
- A reference back to the Engine Installation & Hello World Verification Guide for any engine that hasn't been re-verified recently
- Known gotchas or version-specific quirks worth double-checking that week
- Every item flagged by the Self-Containment Rule below (art/sound/starter files not otherwise provided)

E. Teacher's Key (one per engine)

A worked answer key, for instructor use only - never handed to students. Exists so class prep doesn't depend on the instructor having time to solve every engine's version of the exercise from scratch the week of class. Contains:

- A complete, working Level 1 solution - included purely for the instructor's own quick reference, since Level 1 is usually a small subset of Level 2's solution anyway
- A complete, working Level 2 solution - full code, or the full event/behavior/Blueprint recipe for visual-scripting engines
- Every Level 3 idea listed on that engine's Student Handout, fully worked out end-to-end - not just one, and not just described. The number of engines and the mix of students interested in Level 3 both vary and aren't fully knowable in advance, so "pick one to fully solve, leave the rest as notes" isn't reliable instructor prep. Working all of them also doubles as validation that every suggested idea is actually achievable with the tools/version on hand before it's handed to a student.
- Brief notes on why each solution works, so it can be explained to a student who asks, not just pasted

Rule 3: Every Lesson Is Self-Contained

No lesson should require a student to search for or download something mid-class - that wastes lab time and, especially in the earlier lessons, is a ready-made distraction. Before finalizing a lesson's Teacher's Checklist, audit the Student Handouts and Teacher's Keys for anything referenced but not directly supplied - placeholder art, sound effects, starter/template project files, reference images, and so on.

How to apply this

- If an asset is genuinely needed (e.g., a placeholder sprite for an engine that doesn't ship one), generate or source it ahead of time and bundle it directly in the lesson's Assets folder - don't just tell students to go find one.
- If a checklist item can't be pre-bundled (e.g., an account signup), call it out explicitly and give the exact steps, not just "set this up."
- Prefer simple, engine-agnostic placeholder assets (basic shapes/colors) over anything requiring attribution or licensing research - keeps things fast to produce and legally simple.
- This rule applies most urgently to the first several lessons, while students are still forming habits, but is the standard for every lesson all year.

Rule 4: Visual Programming Prefers Purpose-Built Blocks

When a visual-scripting engine offers a specific, well-named block or node for something, use it instead of the generic variable-setting block - even when the generic block would work just as well. A block named for what it does (Jump To Point, Set Sprite, Play Audio, Set Member) reads as its own documentation; a generic one (Assign Variable, Set Variable, Variables > Set Variable) reads as "some value got set somewhere" and forces a reader to already know the underlying variable name to understand it at a glance. This is worth the extra research per engine because it's exactly the readability visual coding is supposed to offer over text - a block graph that still requires knowing variable names under the hood has given up part of the point.

How to apply this

- Before writing a visual-path step that sets a built-in engine property, check whether that engine has a dedicated block for it first. GameMaker examples found so far: Jump To Point (Movement category) for relative x/y movement instead of Assign Variable on x/y; Set Sprite (Instance category) for a sprite swap instead of Assign Variable on sprite_index; Set Instance Variable for any of its built-in dropdown list (Score, Lives, Health, Image Blend, Visible, Speed, Direction, and more) instead of the fully generic Assign Variable; If Audio Is Playing / Play Audio / Stop Audio (Audio category) for sound-retrigger logic instead of hand-built checks.
- The same principle applies in every visual-scripting engine this course uses, not just GameMaker - Unity Visual Scripting's Set Member (a real, type-checked field on an actual component) over the generic Variables > Set Variable node is the confirmed example so far. Research each engine's action/node library for the same pattern before writing new visual-path content in GDevelop, 001 Game Creator, or Unreal Blueprints; treat an unresearched generic block as a placeholder to revisit, not a finished answer.
- Genuinely custom variables and functions - a student's own moving, enemies_remaining, has_won, blink_timer, or similar - have no dedicated block by definition, so the generic block remains the correct, only choice there. The rule is about built-in engine properties specifically, not a blanket ban on generic variable blocks.
- When a lesson's write-up mixes both cases in one step (for example, a reset routine that touches some built-ins and some custom variables together), say so explicitly rather than defaulting the whole step to the generic block - split the instruction if needed so each variable gets the block it actually deserves.
- Resolved case, found by hands-on testing in Lesson 5: GameMaker's purpose-built property blocks (Set Instance Variable, Set Instance Colour, and similar) each show an

Action Scope dropdown limited to Self, Other, or a specific object asset - which looks, at first glance, like it can't reach one specific, dynamically-created instance held in a variable (e.g. the single enemy a spawn loop just created), since choosing the object asset would affect every existing instance of that object, not just the new one. It first looked like a genuine exception requiring a generic "Assign Variable" block with a dot-notation Name field instead - it is not. An "Instance - Apply To" block, placed right after Create Instance with the If/Else (or whatever needs the scoped target) attached to its RIGHT rather than below it, re-points what "Self" means for everything nested inside it: type the captured Target variable (e.g. new_enemy) into the Expression field at the top of Apply To's own Action Scope pop-up, and every purpose-built block nested inside now applies to that one instance. The purpose-built block was reachable all along - Apply To was the missing piece, not a generic workaround.

Rule 5: Engine Deviations Require a Driving Reason

Every lesson defines one standard form of each exercise, shared across all engines wherever technically possible. A given engine's content should only diverge from that standard form when there is a specific, articulable reason to do so - not because a substitute is merely "close enough," and not because it is structurally similar to the real thing. In a multi-engine class, consistency across engines is worth a lot on its own: it lets one set of instructions, diagrams, and talking points serve every student regardless of which engine they picked, keeps whole-class demonstrations comparable, and keeps the Teacher's Script valid no matter which engine a given student is working in that day. That value doesn't live inside any single engine's file, so it's easy to discount when solving one engine's content in isolation - which is exactly why it needs to be a named rule.

How to apply this

- Only two reasons justify a deviation. First, technical deficiency: the engine genuinely cannot do the standard thing, or cannot do it without skills or setup well beyond the lesson's level. Second, engine-norm mismatch: following the standard form exactly would force a large, unnatural work-around that fights the engine's own idioms, burning lesson time on a workaround unrelated to what's actually being taught that week.
- "Close enough" is not a reason. If an engine can do the standard thing, and doing so does not require an unreasonable amount of adaptation, do the standard thing - even when a structurally-similar substitute would also work and would be less effort to write up. Example: Unreal Blueprints can create a Material with an actual imported placeholder-sprite Texture wired to Emissive Color; a flat, hand-picked color Material is not a valid substitute for that just because both are "a Material swapped via Set Material" - the standard form is specifically the supplied sprite asset, not merely something visually equivalent to it.
- When a genuine deviation reason exists, document it inline in that lesson's content (the Setup step or a trouble bullet) so a future reader can see why that engine differs at a glance, not just that it does.
- When auditing existing content and finding a deviation that doesn't actually meet either bar, fix it rather than leaving it because it already shipped - a shipped shortcut is still a shortcut.

Rule 6: Beyond Level 3 - SuperStretch, Refinement, and Style

The Level 1/2/3 framework gets a working, well-structured version of that week's mechanic into every student's hands, and that's the right scope for the live class. But "working and well-structured" is not the same as "finished being looked at." Real code and real graphs keep improving after they run correctly - a spawn position that's slightly wrong, a sound that's playing from the wrong place, a script that would read better formatted, a graph with the same node called three times when once would do. A lesson that stops at Level 3 without ever naming this leaves students with the impression that working is the finish line. Each lesson/chapter should also cover, beyond Level 1/2/3, its relevant SuperStretch tie-in, a Refinement and Style pass, and - for visual-scripting engines - the graph equivalent of that pass.

How to apply this

- SuperStretch tie-in: where a lesson already has a Lesson_NN_SuperStretch_Ideas.docx, surface the ideas from it that are most relevant to that lesson's specific mechanic, so the connection between "here's the base exercise" and "here's where it can go next" is visible in context, not just filed separately. This does not require creating a SuperStretch document for lessons that don't already have one - those remain optional per Rule 2.
- Refinement and Style: a second-look pass on the lesson's own solution code/graph, covering three things - formatting and readability (consistent indentation, naming, spacing - the code equivalent of showing your work neatly); minor correctness fixes that only show up once you actually run and test the thing, not once you write it (for example, a bullet's initial spawn position being slightly off from the barrel tip); and further separation of concerns beyond what the lesson's base Structure goal required (for example, noticing the firing sound is currently triggered from the player script and moving it to the bullet script, since the bullet is what's actually making the sound). Not every lesson will have all three - some solutions are already clean - but each lesson should be checked for this rather than assumed clean by default.
- Visual-scripting equivalent: for engines built on nodes/graphs rather than text code, formatting becomes node layout and organization (grouping, comments, tidy wire routing - the graph equivalent of consistent indentation), and the DRY principle becomes reusing a computed value or a single method call instead of repeating it (for example, calling Get World Delta Seconds once and feeding that one output wherever it's needed, rather than calling it again at each point of use). Call this out explicitly in visual-scripting lessons rather than assuming the code-side language covers it by analogy.
- This is a standing requirement for lesson content going forward. It does not, by itself, obligate a retrospective retrofit pass on already-completed lessons - see the Book Notes & To-Do document for the scoping discussion on that question.

Rule 7: Visual-Path Steps Are as Explicit as the Code Path

A Student Handout's alternate path for a visual-scripting engine must name the underlying implementation with the same level of detail its code-path counterpart gets: the exact block name and category, the exact field being set on it, and the exact value or wiring that goes there - not just the shape of the logic. "Wire the loop counter into the X position" describes what an experienced coder would already infer; it doesn't tell a first-time GML Visual (or Unity Visual Scripting, or similar) user which field the connector actually goes into, or whether "the loop counter" means a brand-new block dropped on the canvas or something that's already sitting

there. The code path gets exact syntax; the visual path deserves the exact equivalent, not a paraphrase of it.

How to apply this

- Name every block by its exact palette name and category (e.g. "Instance - Create Instance," from the Instances category - not "the create block"), and every field on it by its exact label and the value or expression that belongs there (e.g. Object = obj_enemy, X = 80 + (i * 96)) - the same specificity a reader already gets from a line of typed code.
- When a Level 3 or stretch idea behaves differently in the visual path than a nearby note implies - for example, a spawn-time effect that has to be wired inside the loop itself, not simply reused unchanged from the code path - give it its own explicit, block-by-block sequence rather than asserting nothing changes. This is exactly the gap that surfaced the rule: see the Book Notes & To-Do document for the concrete GameMaker example.
- This applies equally to the Student Handout and the Teacher's Key - a Key that just says "the block equivalent" without naming blocks and fields is exactly as incomplete as a Handout that does the same.
- This is a standing requirement for lesson content going forward. It does obligate a retrospective pass on already-completed lessons' visual-path content, unlike Rule 6's forward-only default - see the Book Notes & To-Do document for the tracked retrofit item.

Rule 8: Slide Decks Carry the Teacher's Script as Speaker Notes

Every lesson's Slide Deck (.pptx) embeds that lesson's Teacher's Script directly in PowerPoint's speaker notes, matched slide-by-slide to the script's timed sections - not just handed out as a separate document. Robert's classroom setup runs Presenter View across two displays (a student-facing screen and his own laptop), which makes the notes pane a live, always-in-view alternative to holding a separate printed script - genuinely more useful there than a paper copy, since it stays in sync with whichever slide is actually on screen.

How to apply this

- Use pptxgenjs's slide.addNotes() on the content slide that corresponds to each of the Teacher's Script's timed headings (e.g. [0:05-0:25] Loops maps onto that section's content slide) - not the section-divider slide that precedes it, which is typically on screen for only a few seconds.
- Copy the script's actual language verbatim, including bracketed stage directions ([Demo cue: ...], [Timing checkpoint: ...], [Pause for guesses]) and quoted dialogue - the notes are the script, not a paraphrase or summary of it.
- Slides with no corresponding Teacher's Script section (title, At a Glance, Agenda, and section-divider slides) don't need fabricated notes - leave them without a notes call rather than inventing narration that doesn't exist in the script.
- The Teacher's Script document itself remains the source of truth and is still built and shipped separately - the speaker notes are a second delivery of the same content for the specific case of presenting live, not a replacement for the standalone document.
- This is a standing requirement for lesson content going forward, starting with Lesson 5's rebuild this session. It does obligate a retrospective pass on already-completed lessons'

slide decks, unlike Rule 6's forward-only default - see the Book Notes & To-Do document for the tracked retrofit item.

Rule 9: Activity Diagrams and Engine Node Diagrams Are a Standing Requirement

Lesson 5 piloted two kinds of diagram beyond the Lesson 2-era Game Loop/Sequences-vs-Events set: a generic, engine-agnostic Activity Diagram of that week's core logic, and engine-specific node/block diagrams for the engines whose visual-scripting path needed one. Both are now a standing requirement, not a Lesson 5-only experiment - this rule formalizes which document gets which diagram, and for which engines.

How to apply this

- Activity Diagram (generic, engine-agnostic): every lesson's core logic - whatever loop, branch, or state-change idea the lesson centers on - gets one simplified, UML-inspired Activity Diagram showing that logic in the abstract, independent of any one engine's syntax or blocks. This diagram is now a standard part of the Student Handout, the Teacher's Key, and the Slide Deck for every lesson, the same way it was for Lesson 5 - not conditional on whether the lesson feels diagram-worthy.
- Engine node diagrams apply only to engines offering both a script/code path and a visual path in this course - currently GameMaker, Unity, and Unreal. Engines that are visual-only with no code alternative (Scratch, GDevelop, 001 Game Creator) or code-only with no visual alternative (Godot, TIC-80) don't have a dual-track contrast to diagram, so this part of the rule doesn't apply to them.
- Teacher's Key: always gets an engine-specific node diagram for that lesson's Level 1 solution, and a complete combined-sequence diagram covering the full worked Level 3 solution when the visual path is complex enough to benefit from seeing every node/block wired together at once (Lesson 5's GameMaker and Unreal Keys are the first examples).
- Student Handout: always gets an engine-specific node diagram to accompany the Level 1 requirements, for any engine with both a script and a visual option - this is unconditional, not tied to novelty. Beyond Level 1, the Handout gets an additional node diagram only when something genuinely new or unusual is being introduced that a text description alone would leave ambiguous (Lesson 5's GameMaker "back door" wireframe is the model case - a real gap in the block palette that needed a close-up illustration, not just a Level 1 overview).
- Sidebars and Slides: add a diagram to either only when it actually helps enhance or clarify the surrounding text - never as a fixed per-lesson requirement the way the Activity Diagram and Handout/Key node diagrams are.
- An engine-specific node diagram must authentically reflect how that engine's own visual editor actually looks and behaves - GameMaker's block-stack style (solid blocks, fields as plain text rows, no explicit pins) is genuinely different from Unity/Unreal's node-graph style (header-bar nodes, exec and data pins, wires connecting sockets), and a diagram should never borrow the wrong engine's visual language for the sake of consistency. Because of this, a Slide Deck's diagram of the same underlying idea may not exactly match an engine-specific document's diagram of it - the Slides version can simplify or generalize in ways an engine-specific Handout or Key diagram should not, since the engine-specific version's whole job is to show that engine's actual paradigm.

- This is a standing requirement for lesson content going forward. Like Rule 7 and Rule 8, it does obligate a retrospective pass on already-completed lessons - see the Book Notes & To-Do document for the tracked retrofit items covering both past lessons (1-4, 6-15) and the plan for applying this rule as future lessons (16-30) are authored.

Rule 10: Lecture-Demo Engine Rotates - It Does Not Default to Whatever Was Used Last Week

The Syllabus, the Engine & Platform Guide, and the Engine Showcase all promise students that each week's live lecture demo rotates through the course's supported engines rather than sticking to one - and the original 30-Lesson Master Schedule was built with an actual Featured Engine and Coding Style column modeling that balance. That promise was never enforced as a rule, and because every lesson's demo cue is written as "continue the same demo project from last week," nothing stopped one engine from quietly becoming the permanent default once it got picked - which is exactly what happened: Godot was the featured lecture-demo engine for every lesson from 3 through 14 before this was caught and fixed.

How to apply this

- Every lesson's Plan document names its lecture-demo engine in an explicit "Featured Engine (lecture demo)" line - this is the single authoritative record of which engine that lesson's Teacher's Script demo cues must use. Check it before writing or editing a demo cue; don't infer the engine from what a nearby lesson happened to use.
- For now, the rotation draws only from the engines and coding styles actually in active use by enrolled students - currently Scratch, GameMaker (GML script), GameMaker (GML Visual), Unity (C#), and Godot. Lessons 5 through 14 use each of these five exactly twice, with no two consecutive lessons repeating the same choice: L5 GameMaker Visual, L6 Scratch, L7 Godot, L8 Unity C#, L9 GameMaker script, L10 Godot, L11 Unity C#, L12 GameMaker Visual, L13 Scratch, L14 GameMaker script. Where GameMaker appears twice, the second occurrence is written as an explicit callback to the first (e.g. Lesson 9's script-mode demo calls back to Lesson 5's Visual-mode demo on the same underlying project), mirroring the original schedule's same-engine-two-modes pairing. Lesson 15 remains demo-free by design - it's a workshop/showcase day, not a lecture.
- A demo cue must be technically accurate to its lesson's actual assigned engine - exact variable names, method/function names, event names, or block/category names for that engine - not a relabeled copy of whatever the previous engine's cue said. When a lesson's assigned engine changes, its Teacher's Script demo cues, its Plan's Featured Engine field, and (per Rule 8) its Slide Deck speaker notes must all be rewritten from that engine's own actual Handout/Key content, not just find-and-replaced.
- This restricted 5-choice pool is an intentional, temporary narrowing, not the course's permanent design - the full rotation is meant to eventually cover all 8 engines and all 3 coding styles once more students are using the currently-idle ones. See the Book Notes & To-Do document for the tracked item to expand it.
- This is a standing requirement for Lessons 5 onward. It does not obligate a retrofit of Lessons 1-4 (Lesson 1 has no demo by design; Lesson 2 is Scratch; Lessons 3-4 predate this rule but happen to already be Godot, an allowed choice) - but any future rebuild of Lessons 3-4's demo content should still record their engine in the Plan's Featured Engine field per this rule.

Rule 11: Meaningful Variable Names Over Generic Counters

A loop counter named i, j, or k is a habit carried over from decades of textbook examples where the counter genuinely means nothing beyond "which pass is this" - but in this course's actual lessons, that counter almost always stands for something a student can name: which enemy is being spawned, which wave this is, which item in an inventory is being checked. Naming it enemy_count, wave_index, or item_index instead costs nothing at read time and pays for itself the moment a lesson's code or blocks get more complex than a single, self-contained loop - a reader (or a student six lessons later, reusing the pattern in a bigger project) shouldn't have to hold "i means the enemy counter, right here, in this one loop" in their head as unstated context.

How to apply this

- Prefer a name that says what the value actually counts or tracks - enemy_count, wave_index, hit_count - over a bare single letter, in every engine and every coding style: typed code, GML/GDScript/C# variables, and GML Visual/Unity Visual Scripting/Blueprint block fields alike. This applies to the visual tracks just as much as typed code - a "Set Variable" block's Variable field is just as much a variable name as a line of text is, and deserves the same care.
- This is specifically about loop counters and similar tracking values with an actual, nameable meaning in context - not a blanket ban on short names. A truly generic, throwaway index with no meaning beyond position (rare in this course's lessons, but possible in a genuinely abstract utility function) doesn't need to be stretched into an awkward descriptive name just to satisfy this rule.
- When the same counter is used identically in both a Code path and a Visual path for the same engine (GameMaker's dual-track lessons are the main case), use the exact same name in both - this is what actually lets a student compare the two side by side and see they're doing the same thing, the same way this course already insists on matching captured-instance variable names (new_enemy) between GameMaker's Code and Visual paths.
- This rule was adopted after Lesson 5's GameMaker content was found using i for the spawn loop's counter and inst for the captured instance in its Code path, while the Visual path (built to match) already used the more descriptive new_enemy - inconsistent with itself, not just with this rule. Both paths were corrected to enemy_count and new_enemy together; see this lesson's Book Notes entry.
- This is a standing requirement for lesson content going forward, starting with Lesson 5's correction. It does obligate a retrospective pass on already-completed lessons using generic i/j/k counters across other engines - see the Book Notes & To-Do document for the tracked retrofit item.

Rule 12: Common Ground Across Engines - Consistent Naming and Code Structure

Beyond Rule 5's requirement that every engine implement the same standard exercise, and Rule 11's requirement that variable names be meaningful rather than generic, this course treats the specific name chosen and the shape of the solution itself as something to keep as consistent as possible across all 8 engines and all 3 coding styles - not just conceptually equivalent, but recognizably the same when placed side by side. This was not enforced strictly in Lessons 1-4, and only partially in Lesson 5 (Rule 11's naming fix was applied engine by

engine as each was found, not as a single coordinated cross-engine pass). It has become clear this term that this consistency is not a cosmetic nicety - it is one of the load-bearing reasons the multi-engine structure works as a single shared class rather than eight parallel ones. Students working in different engines, and even different coding styles within the same engine, have been observed helping each other across that divide - a student in one engine comparing notes with a student in another on the same week's exercise - and the more common ground the actual names and structure share, the easier that cross-engine collaboration is. A student explaining their enemy_count variable to a classmate whose engine calls the identical concept enemy_number, or whose spawn loop is organized in a different order for no engine-specific reason, has to translate before they can actually help - friction this course can design away. This is now treated as a fundamental precept of the multi-engine design, not an optional polish step.

How to apply this

- When the exact same concept appears in multiple engines' solutions for the same lesson - a loop counter, a spawned-object reference, a life/score variable, a state flag - use the exact same underlying name across every engine, not just a meaningful name chosen independently per engine. Rule 11 requires a meaningful name; this rule adds that the specific word chosen should match everywhere it applies, engine syntax permitting (snake_case vs. camelCase still follows each language's own convention, but the underlying word - enemy_count, not enemy_count in one engine and enemy_number in another - should agree).
- Where an engine's own existing code or conventions create a real naming conflict - for example, Unity's Lesson 5 EnemySpawner already has a public enemyCount field for the total number to spawn, which would collide with also naming the loop counter enemyCount - document the conflict and the reasoning behind whatever name was chosen instead, rather than silently picking something different with no explanation. See the Book Notes & To-Do entry on Unity's enemyCount/enemyIndex naming for the model case; it is deliberately left unresolved for now pending a coordinated cross-engine pass, not fixed in isolation.
- Structure the solution's code or graph the same way across engines wherever each engine's own idioms allow it - the same order of operations, the same separation of concerns, the same which-object-does-which-job - not just matching variable names wrapped around a differently-organized solution. This is Rule 5's engine-deviation standard applied one level deeper: consistency isn't only about which exercise gets built, but how its actual solution is organized.
- This applies across coding styles within a single engine just as much as across engines - a Code-path and a Visual-path solution for the same engine should use matching names and matching structure. This course already requires matching names for GameMaker's dual tracks under Rule 11; this rule extends the same expectation to structure, and to every other dual-track engine.
- This is a standing requirement for lesson content going forward. Unlike Rule 6's forward-only default, it does obligate an eventual retrospective pass reconciling naming and structure across all 8 engines for already-completed lessons - starting with Lesson 5, where GameMaker and Godot now use enemy_count/new_enemy, Scratch independently settled on enemy_number before this rule existed, and Unity, Unreal, GDevelop, 001 Game Creator, and TIC-80 have not yet been fully retrofitted. This dedicated cross-engine reconciliation pass is deliberately scheduled for later rather than done immediately - see the Book Notes & To-Do document for the tracked item, which

runs alongside (but is broader than) the individual per-engine Rule 11 naming retrofit already underway.

Rule 13: Shared State Lives on a Dedicated Manager Object

Lesson 6 introduces the course's first genuinely shared piece of state - how many enemies remain - and only Unity's content routed it onto a new, dedicated object built for exactly that job (a GameObject literally named GameManager). Every other engine bolted the same counter onto whatever object was already at hand: Godot's main scene, GameMaker's spawner, Scratch's own hit-detection branch, Unreal's Level Blueprint. This was caught during a Lesson 7 review, not planned - the same pattern Rule 11 and Rule 12 emerged from. Robert's judgment, once the inconsistency was pointed out: this isn't a cosmetic preference, it's a real architectural habit worth teaching deliberately, because "who owns this piece of state" only gets harder to answer cleanly as a capstone grows across the year. Retrofitted into Lesson 6 the same week it was caught, with a callout in Lesson 7 so students already past that lesson know to revisit their own projects.

How to apply this

- When a piece of state is shared across multiple objects - a win/lose counter, a score, lives, a wave number, anything more than one object needs to read or change - give it a dedicated object built for that job, not a variable bolted onto whichever object happened to be doing something else nearby. The objects that report to it (an enemy announcing its own destruction, say) should only ever report; they shouldn't also count, decide, or draw anything themselves.
- Name that dedicated object GameManager (or the engine's own idiomatic equivalent of that name) across every engine that has an object/actor/node system: a Sprite in Scratch, a Node in Godot, an object in GameMaker, an Actor Blueprint in Unreal. Keeping the same name is itself part of Rule 12 - it's one more concept a student on one engine can point to and have a student on a different engine immediately recognize.
- This is a real exception, not a loophole: an engine with no object/actor system at all has nothing to dedicate a manager object to, and forcing one into existence would fight the engine's own idiom rather than follow it (Rule 5's engine-deviation standard, applied here). GDevelop's scene variables and TIC-80's global Lua variables are already global, unowned by any one object, and already serve exactly the role a manager object would - so neither gets one. 001 Game Creator is a narrower case: its Actor Triggers are bound to the actor they're attached to, with no verified mechanism for a separate Actor to react to another Actor's destruction, so its win-tracking logic stays on the enemy Actor Template itself rather than moving to an invented feature. All three exceptions are documented inline in their own Lesson 6 content, not left silent.
- This is a standing requirement for lesson content going forward, starting with Lesson 6's retrofit. Future lessons introducing a new piece of genuinely shared state (score, lives, wave number, and so on in later lessons) should route it onto the same GameManager object rather than creating a second dedicated object per concept - one shared-state home per project, not one per feature.

Rule 14: Code Completeness by Level - Handouts and Keys

Students and the teacher need different things at each level, and a partial code box is worse than none: a student who copies half of what is needed gets a project that silently does not

work, and a teacher checking work against an incomplete Key cannot tell a correct solution from a broken one. Every lesson therefore follows one completeness standard, for every engine and every coding style.

How to apply this

- Student Handout, Level 1: complete code for every method (event, function, script, or block sequence) that the lesson adds or modifies - shown whole as it should stand at the end of the lesson, never as a fragment of just the new lines. That includes initialization (Create/Ready/Start events and variable setup) and display (Draw GUI, OnGUI, labels) whenever the lesson touches them. Methods the lesson does not add or change are not repeated. Showing only the new lines has repeatedly left students unsure where those lines go and what surrounds them. Highlight the new lines inside each listing (yellow highlight in Word/PDF; where a medium cannot highlight, a trailing // NEW comment) so the week's change is easy to spot.
- Student Handout, Levels 2 and 3: descriptive prose, plus diagrams where a diagram clarifies (per Rule 9). Students are expected to work these out themselves, using what they learned in Level 1; the Handout describes the goal and any engine-specific hint, never the finished code.
- Teacher's Key, all three levels: complete code. Level 1 as in the Handout, Level 2 as the full final code (or an explicit statement that no lines change, with the specific checks to run), and Level 3 as the complete combined code for all of that level's goals together, so the teacher can compare a student's project against a working whole rather than assembling fragments from talking points.
- Applies to every coding style: typed code, and the block, node, or event-sheet equivalent of it (every block with its exact field values and Target, per Rule 7). Applies to every engine, including the visual-primary ones.
- Check before shipping: for each engine, mentally run only what the Handout shows from a fresh copy of last week's project. If anything the lesson needs to run - initialization, drawing, wiring - is not on the page, the Handout is incomplete.

Rule 15: Explicit, Beginner-Friendly Names

Every name a student has to type or recognize - variables, functions, signals, nodes, objects, assets - must say what the thing is in plain words, chosen for someone who has never used the engine, not for a veteran who already knows its jargon. A name that is obvious to an experienced developer can still mislead a newcomer: one student read WinLabel as "Window Label" because "Label" is Godot's technical term for a text-display node, which means nothing to a beginner. The course uses WinText instead, and the same name in every engine that has such an object (Rule 12).

How to apply this

- Name things by what they show or do in the game: WinText, LoseText, ScoreText, LivesText, WaveText, enemies_remaining, on_enemy_escaped. Prefer a short plain phrase over an engine term of art (Label, Node, Actor, Pawn, Controller) when the engine term is the only thing making the name unclear.
- Never abbreviate where a full word fits (enemies_remaining, not enm_rem), and never use bare single letters (Rule 11).

- When an engine's own term must appear (the node type is still a Label in Godot), say so once in the instructions - "add a Label node and rename it WinText" - so students connect the engine term to the plain name, then use only the plain name from then on.
- Where a name is fixed by the engine (a built-in signal, an engine function like _process, GameMaker's __dnd_lives), explain it once the first time it appears rather than hiding it.
- Check before shipping: read every new name aloud to someone who has not used the engine. If they would guess wrong about what it is, rename it.

Final Deliverable: The Complete Script Document

Because each lesson shows only the methods it adds or modifies, no single lesson shows a whole working program. Both the finished course and the book therefore end with a Complete Script Document: one reference per engine and coding style containing the entire finished Semester 1 project - every object, script, event, function, node graph, or block sequence, in full, organized by object and in the order a student would build it, with the lesson in which each piece was introduced noted beside it. It is generated from the same lesson build scripts so the two can never drift apart, and it serves as the answer key for "my project is broken and I cannot tell which piece is missing."

Semester 2 Addendum: Testing Becomes Ongoing

Semester 1 has no dedicated testing/playtesting lesson - each week's GDD tri-state checkbox (Plan to Implement / Implemented / Tested) plants the vocabulary and the honest habit of noticing when "tested" lags behind "implemented," but that's deliberately as far as it goes while students are still learning fundamentals across 8 engines. Lesson 15 closes Semester 1 by naming this directly and framing testing as something that becomes a regular, ongoing part of the process starting next term.

When building Semester 2 lessons (16 onward), integrate testing into the regular workflow throughout, rather than concentrating it into a single dedicated lesson. It can stay informal and manual - a student playing their own game with a critical eye, or a quick structured swap with a classmate - but it should show up as a recurring habit across multiple lessons' Lab Time Notes and Level 2 checklists, not just Lesson 27's existing "Playtesting Workshop" slot. That lesson remains the natural place for a more structured, whole-class version once the capstone is feature-complete; it doesn't need to be the only place testing is mentioned.

Visual Branding & Templates

Every document and slide in this course shares one visual identity: navy and gold, the SLHE crest, Cambria headings on slides, Arial body text in Word docs. There are two ways to work within it, depending on who's doing the work:

- **Building something by hand in Word or PowerPoint:** start from SLHE_VGC_Word_Template.dotx or SLHE_VGC_PowerPoint_Template.potx, both in this Lesson 00 folder. Opening one creates a new, untitled file with the styles/layouts already built in - the template file itself is never edited directly. SLHE_VGC_Template_HowTo_Guide.docx (also in this folder) walks through exactly how, written for someone who isn't a regular Word/PowerPoint user.

- **Generating a full document or deck programmatically:** the way every lesson so far has been produced - every build script requires brand.js, the shared module holding the exact colors, fonts, and header/footer/crest logic. Never hardcode a hex color or rebuild the header from scratch in a new script; require brand.js and call its helpers, the same way every existing script does.

The complete visual reference - exact hex colors, logo clear-space, table styling - is Brand_Style_Guide.docx in the Lesson 00 - Standards folder, alongside the templates. The source images behind all of it (crest/logo, icon set, slide backgrounds, diagram SVGs) and every build script live in Brand_Assets/ at the root; Brand_Assets/Build_Scripts/README.md explains how to re-run any of them to regenerate or extend a document.

Slide Deck Conventions

Both slide decks so far and the PowerPoint template share one canvas size (13.333 x 7.5in, PowerPoint's Widescreen) and the same four slide layouts:

Layout	Background	Used for
Title Slide	Navy, crest centered	The deck's opening slide - lesson title, one-line description, date.
Section Divider	Navy	Marks a new agenda segment (e.g. "Sequences," "The Game Loop") - a visual "we're moving on" cue.
Content	Light	The workhorse slide - a title plus body text, bullets, or images. Most of any deck is this layout.
Closing	Navy, crest centered	"See You Next Week," with a one-line pointer to next lesson's exercise.

- SLHE_VGC_PowerPoint_Template.potx packages these four as real, selectable PowerPoint layouts (Home > Layout) - use it as the starting point for a new deck rather than rebuilding the masters from scratch.
- Optional, skippable material - worth covering if lecture is running ahead of schedule, but not required to hit the lesson's objectives - is marked with a gold "BONUS" pill instead of the usual time-tag, and flagged the same way in the Teacher's Script with a bracketed time-permitting note. The Lesson 2 Activity/State diagram slides are the first example of this pattern; reuse it for any future optional content rather than inventing a new convention.

Diagrams

Starting in Lesson 2, concepts that are easier to grasp visually get a simple diagram instead of relying on text alone: the Game Loop, Sequences vs. Events, and - as the first piece of optional bonus content - a simplified, UML-inspired Activity Diagram and State Diagram. Class Diagrams are intentionally deferred to a later lesson, once students have enough object-oriented vocabulary for one to mean something.

- Every diagram is built as an SVG source file, then rendered to PNG for embedding - both live in Brand_Assets/Diagrams/. Keep building new diagrams the same way (editable source plus a rendered copy), not as a flat image with no source.
- Known rendering gotcha: the SVG-to-PNG renderer used throughout (cairosvg) does not compute arrow direction correctly for marker-based arrowheads on curved paths - they'll all point the same direction regardless of the curve's actual direction. Work around it by drawing the arrowhead as a manually-rotated triangle instead of a marker, with the rotation angle computed from the curve's tangent at its endpoint. Any of the four existing diagram SVGs shows the pattern.
- Introduce a new diagram type only when it earns its place - when a concept is genuinely easier to show than to describe - not on a fixed schedule.

File Organization

The course folder has three tiers: the source library everything is built from at the root, then inside Lessons/, one shared-reference layer plus one subfolder per lesson.

PDF Companions

Every .docx built for this course ships with a matching .pdf, generated from the same file via LibreOffice (soffice --headless --convert-to pdf) and placed alongside it. The PDF, not the .docx, is the version posted for students to print and keep - it opens identically on any device without a Word-compatible app. The one exception is Lesson_NN_GDD.docx: because it's a living, fill-in document students and instructors edit directly, the .docx remains the primary, distributed format. A .pdf is still generated for it as a reference snapshot, just not treated as the version to hand out.

Root

- Brand_Assets/ - source images and build scripts (see below)
- Lessons/ - the standards/templates folder, the Game Design Document template, and one subfolder per lesson (see below)

Lessons/ (top level)

- Game_Design_Document_Template.docx - the student project template, sitting alongside the lesson folders rather than inside any one of them since it's used across multiple lessons and both semesters
- Semester_01_Capstone_GDD.docx - the instructor-facing master design for the Semester 1 compulsory mini-game (High Concept, full Level 1/2/3 feature breakdown, Feature-to-Lesson Map). Not distributed to students - each Lesson_NN_GDD.docx's own Goals table is a slice of this document, revealed to students incrementally as their lessons reach each feature. Sits at this level for the same reason as the student template: referenced across many lessons, owned by none of them.
- Lesson 00 - Standards/ and each Lesson NN - Topic Name/ folder (below)

Lessons/Lesson 00 - Standards/

- This document
- Brand_Style_Guide.docx - the full visual reference, kept with the templates since they're used together

- SLHE_VGC_Word_Template.dotx, SLHE_VGC_PowerPoint_Template.potx, SLHE_VGC_Template_HowTo_Guide.docx - the manual, hand-editable templates and their how-to guide

Lessons/Lesson NN - Topic Name/

- Lesson_NN_Plan.docx
- Lesson_NN_Teacher_Script.docx
- Lesson_NN_Slides.pptx
- Lesson_NN_Prep_Checklist.docx, Handouts/Lesson_NN_Handout_[Engine].docx (up to 8), Keys/Lesson_NN_Key_[Engine].docx (up to 8, instructor-only), and Assets/ (bundled placeholder art/sound/starter files) - these four appear starting with the first lesson that has a per-engine hands-on exercise. Lesson 1 doesn't have them because it's whole-class introductory content with no hands-on exercise yet - that's expected, not a gap to fix. Lesson 2 onward, plan on all four being present.
- Lesson_NN_SuperStretch_Ideas.docx (optional) - a generic, engine-agnostic curiosity list of ideas beyond that lesson's Level 3, for a student who finishes early or wants a sense of what's possible further out. Not one of the five required outputs, not required for every lesson, and explicitly exempt from the Self-Containment Rule below - unlike everything else in the lesson, a student chasing one of these ideas is expected to look things up on their own. Add one when a lesson's cut Level 3 ideas and other "could be fun" material add up to enough for a worthwhile list.
- Lesson_NN_GDD.docx (Lessons 1-15 built so far) - a pre-compiled, lesson-specific version of the Game Design Document so students and instructor don't have to assemble it live each week. Lesson 1's is a condensed, non-fill-in "sneak peek" (Level 1/2/3 framework, the real 30-lesson roadmap, the Five Disciplines) handed out at the end of class as a preview - it does not replace the real template. Lesson 2 onward, it's the full Game_Design_Document_Template.docx content plus a "Lesson N Goals" tri-state checkbox table (Plan to Implement / Implemented / Not Planned) inserted into Levels / Structure, pre-filled with that lesson's own non-engine-specific Level 1/2/3 goals so students can track them without copying by hand. Only build a new one when a lesson actually needs its own goals tracker or other lesson-specific content beyond the shared template - otherwise pointing students at Game_Design_Document_Template.docx directly is fine.

Brand_Assets/

- Logos_and_Crests/, Icons/, Slide_Backgrounds/, Diagrams/ - reusable source imagery, PNG plus an editable SVG wherever one exists
- Build_Scripts/ - the script that generates every document and deck above, brand.js, the OOXML post-processing utilities (fix_content_types.py, make_dotx.py, make_potx.py), and a README explaining how to re-run any of them

Revision Note

This is a living document. If the output structure or the three-level framework needs adjusting once real lessons are in front of real students, update this document first, then apply the change going forward - earlier lessons do not need to be retrofitted unless something is actually broken.

This revision (after Lessons 1 and 2 were both complete) folded in the branding, template, slide-deck, and diagram conventions that emerged from actually building them, on top of the original

three rules. Expect another pass like this once the class has run for real and we can see how students actually respond - treat everything above as the current best answer, not a fixed spec.